

DA-IICT

Software Engineering – IT632

HOSTEZY

Smart Hostel & Mess Management System

Design Models Document

Date: 15th March 2026

Team Members:

202512015 – Trivedi Jaimin Pratikkumar

202512097 – Modi Garv Rajeshbhai

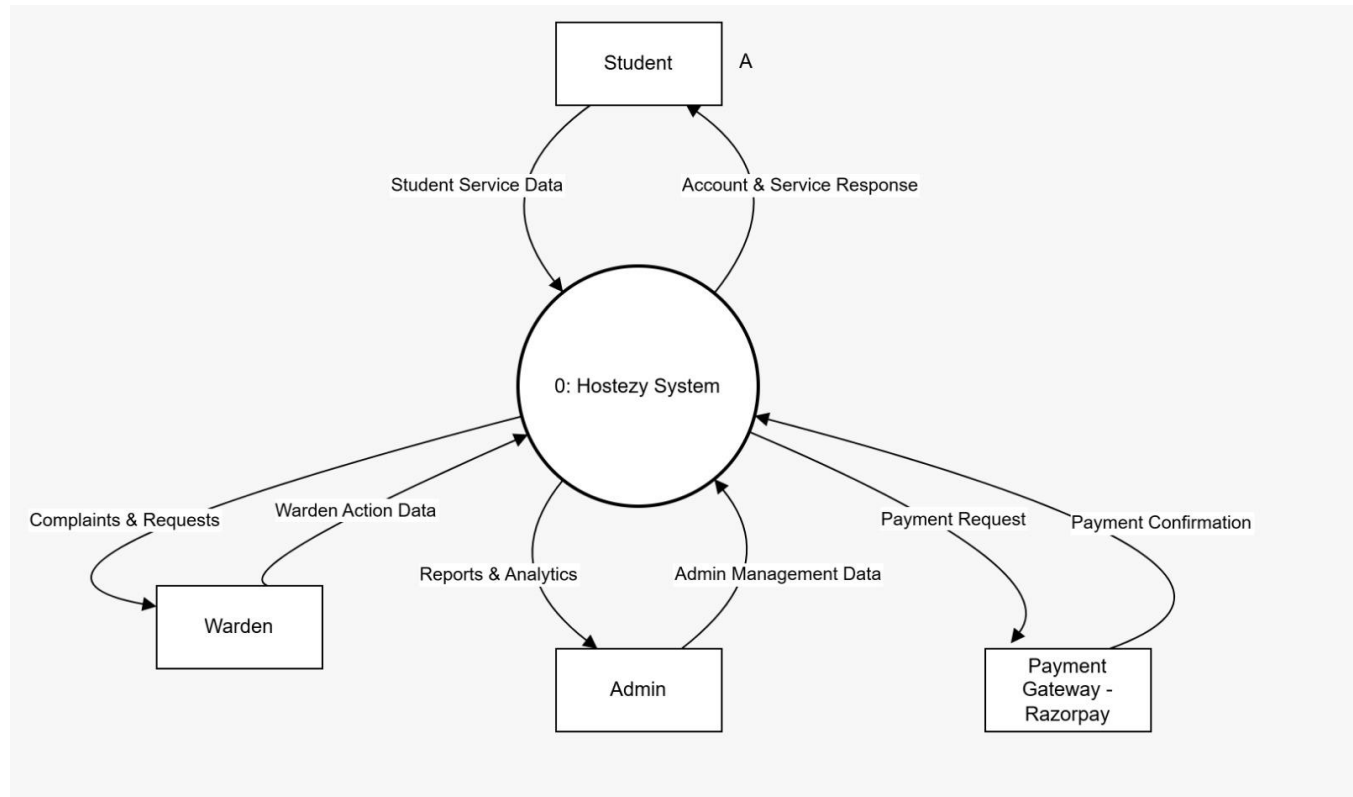
202512065 – Patel Dev Vijaybhai

202512038 – Gandhi Tirth Ashishbhai

1. Data Flow Diagrams (DFD)

Data Flow Diagrams illustrate how data moves through the Hostezy system, showing the transformations applied to data as it flows between external entities, processes, and data stores. The diagrams follow Yourdon-DeMarco notation and are structured across three levels of abstraction.

1.1 Context Diagram (Level 0)



The Context Diagram presents the highest-level view of the Hostezy system as a single process (Process 0), showing only the major external entities that interact with it and the primary data flows between them. No internal detail is exposed at this level.

External Entities:

- Student – The primary end-user who registers, makes payments, books mess subscriptions, and raises complaints.
- Warden Admin – Manages hostel room allocations, reviews complaints, and generates occupancy reports.
- Payment Gateway (Razorpay) – External third-party service that processes all financial transactions securely.

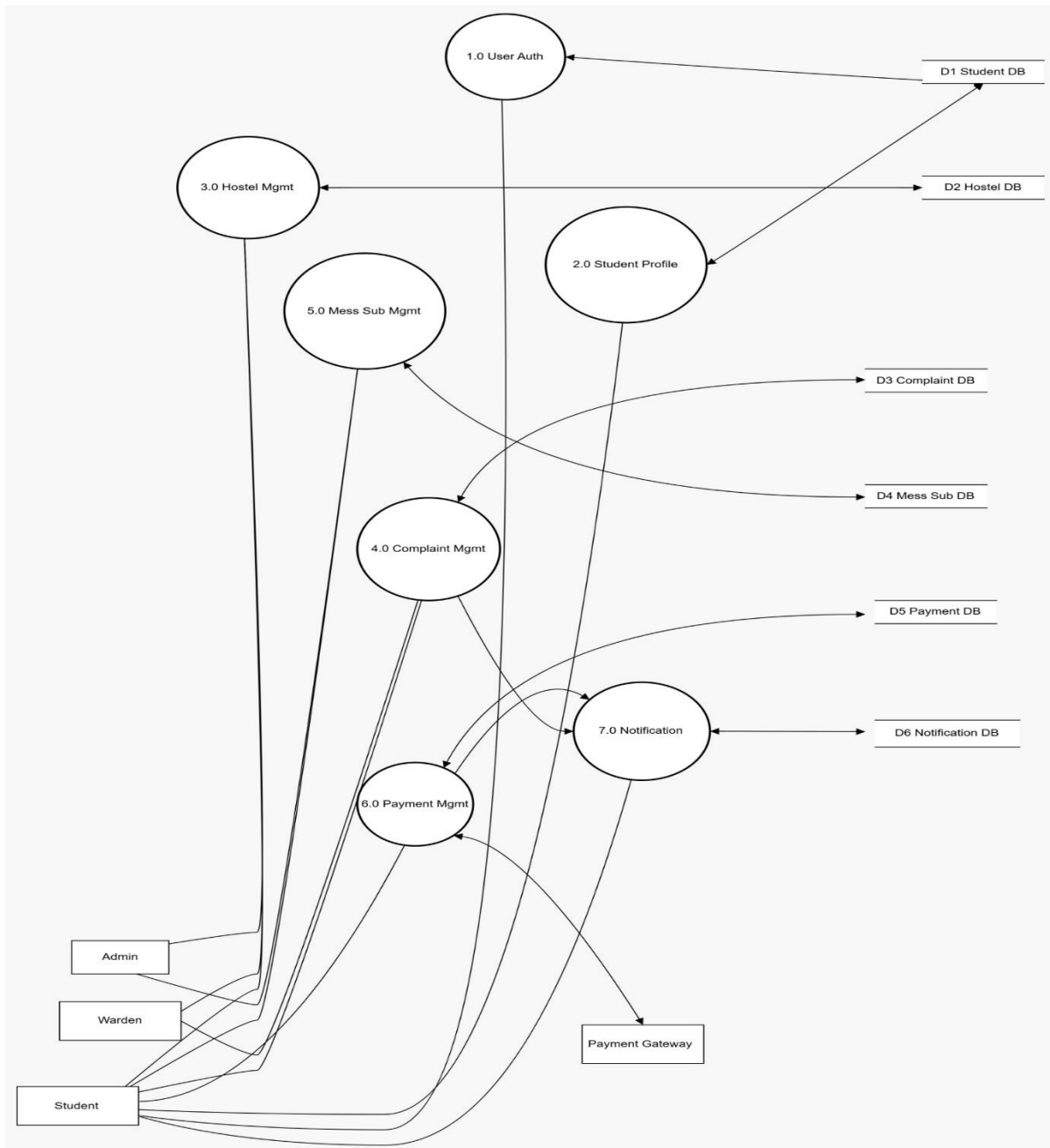
Key Data Flows:

- Student → System: Service requests, login credentials, payment requests, complaint submissions.
- System → Student: Account & service responses, notifications, payment confirmations.
- Warden Admin → System: Action data (room updates, complaint resolutions), management commands.
- System → Warden Admin: Reports, analytics, complaint lists.

- System ↔ Razorpay: Payment requests outbound; payment confirmations inbound.

Design Rationale: The single-process abstraction confirms system boundaries and clarifies which entities are in-scope versus external. This diagram serves as the agreement baseline between stakeholders before any internal design is committed.

1.2 Level 1 DFD



The Level 1 DFD decomposes Process 0 into seven major functional processes (1.0–7.0), each corresponding to a distinct subsystem of Hostezy. Six data stores (D1–D6) are introduced, and all flows are balanced with the Context Diagram.

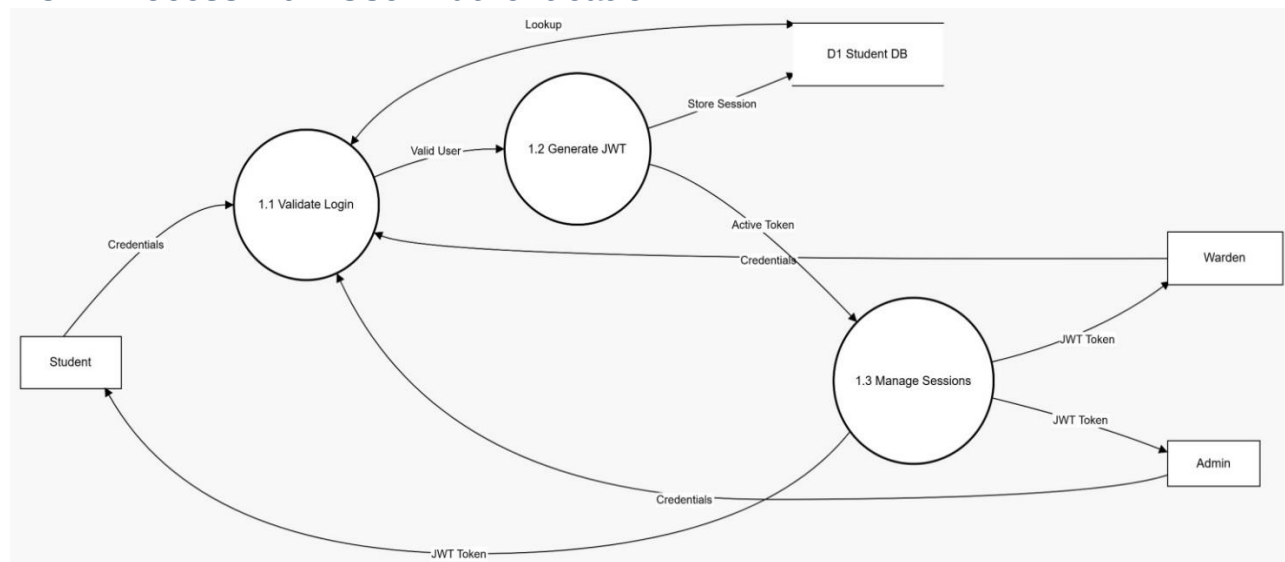
Process	Description
1.0 User Authentication	Handles login validation, JWT token generation, and session lifecycle for all three user roles (Student, Warden, Admin).
2.0 Student Profile Management	Manages student registration, profile updates, and retrieval of room/hostel assignment information.
3.0 Hostel & Room Management	Covers the full lifecycle of room operations: creation, allocation, vacancy tracking, and vacate request processing.
4.0 Complaint Management	Supports complaint submission by students, viewing and resolution by wardens, and automatic notification triggers.
5.0 Mess Subscription Management	Controls mess subscription creation and renewal, daily attendance logging, and menu plan configuration by the Mess Admin.
6.0 Payment Management	Orchestrates end-to-end payment flow: initiation by student, processing via Razorpay, status verification, and record update.
7.0 Notification System	A cross-cutting utility process that generates payment reminders, complaint status updates, and subscription expiry alerts, then logs all alerts.

Data Stores:

- D1 Student DB – Student profiles, login credentials, room allocation records.
- D2 Hostel DB – Hostel details, room information, occupancy and vacancy status.
- D3 Complaint DB – Complaint records, status flags, and resolution information.
- D4 Mess Subscription DB – Subscription plans, student subscriptions, attendance logs, menu data.
- D5 Payment DB – Payment transactions, history, and subscription payment records.
- D6 Notification DB – System alerts, payment reminders, and complaint update logs.

1.3 Level 2 DFDs

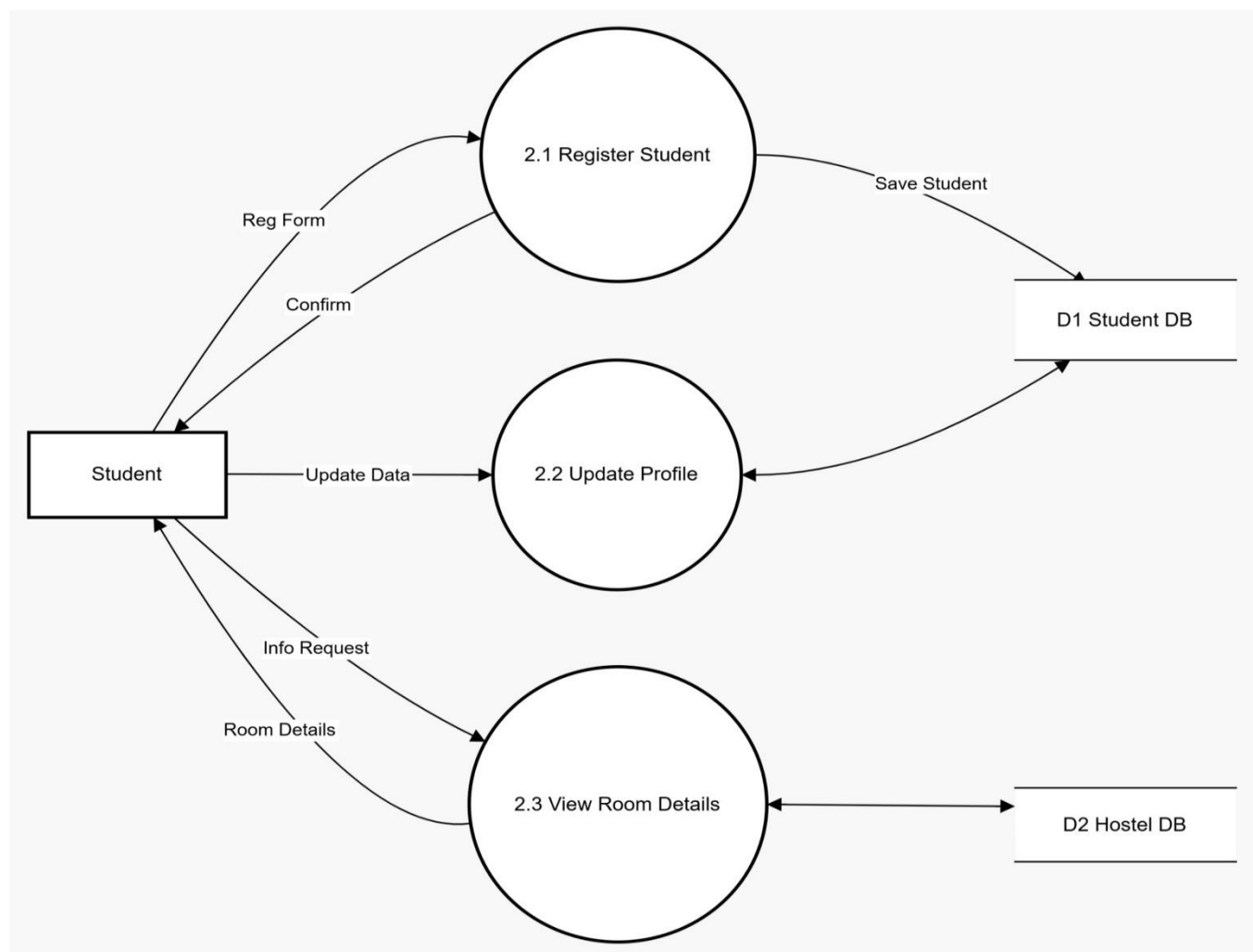
1.3.1 Process 1.0 – User Authentication



Decomposes authentication into three sub-processes covering credential validation, token generation, and session management. Interacts exclusively with D1 Student DB.

Sub-process	Description
1.1 Validate Login	Receives username/password, queries D1 Student DB, and returns a verified user record. Rejects invalid credentials with an appropriate error response.
1.2 Generate JWT Token	Upon successful validation, creates a signed JWT token encoding the user role (Student / Warden / Admin) and session expiry. Returns the token to the client.
1.3 Manage Sessions	Stores active session tokens, handles token refresh, and invalidates expired or logged-out sessions in D1.

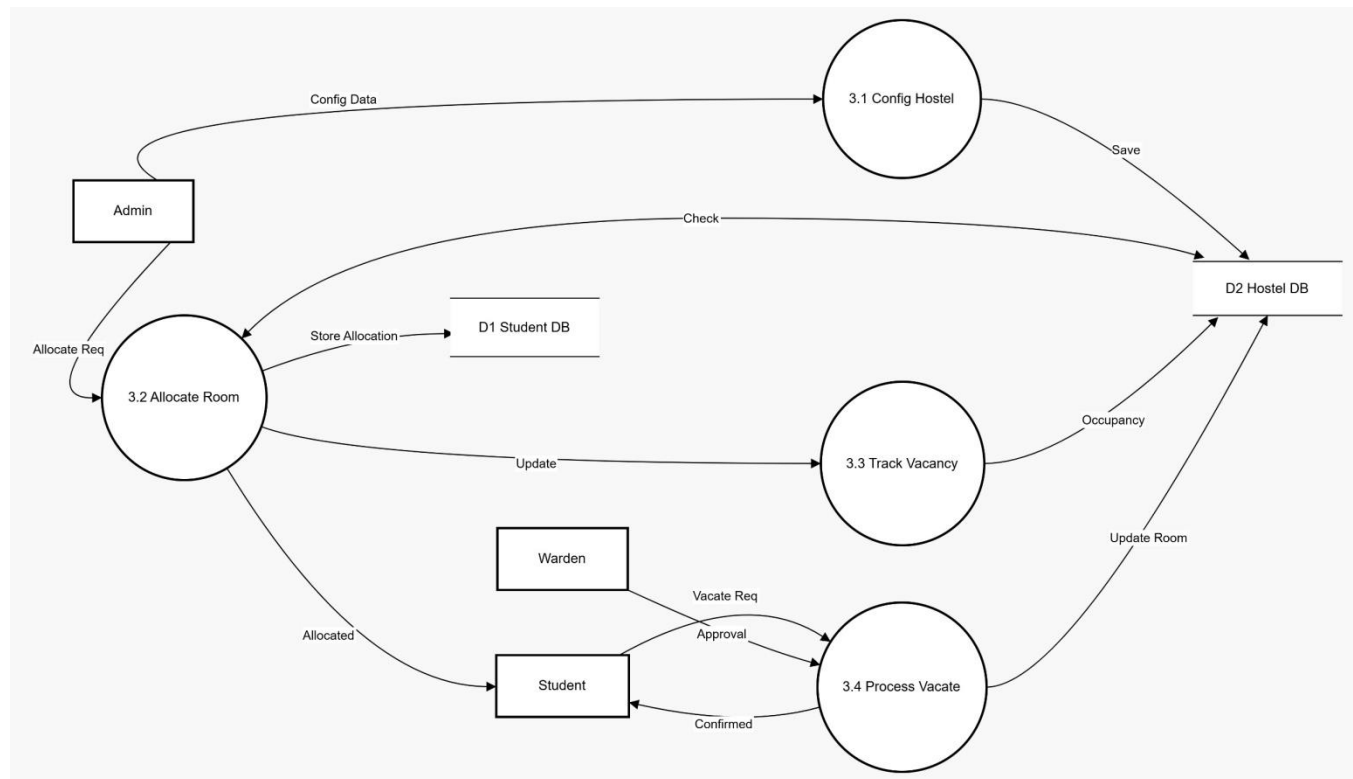
1.3.2 Process 2.0 – Student Profile Management



Handles student data across registration, profile editing, and room detail viewing. Reads from both D1 Student DB and D2 Hostel DB.

Sub-process	Description
2.1 Register Student	Accepts registration form data, validates uniqueness of email and enrollment number, and persists a new student record in D1.
2.2 Update Profile	Allows authenticated students to modify personal details. Writes updated data back to D1.
2.3 View Room / Hostel Details	Queries D2 Hostel DB using the student's allocated room ID and returns room configuration, block details, and warden contact.

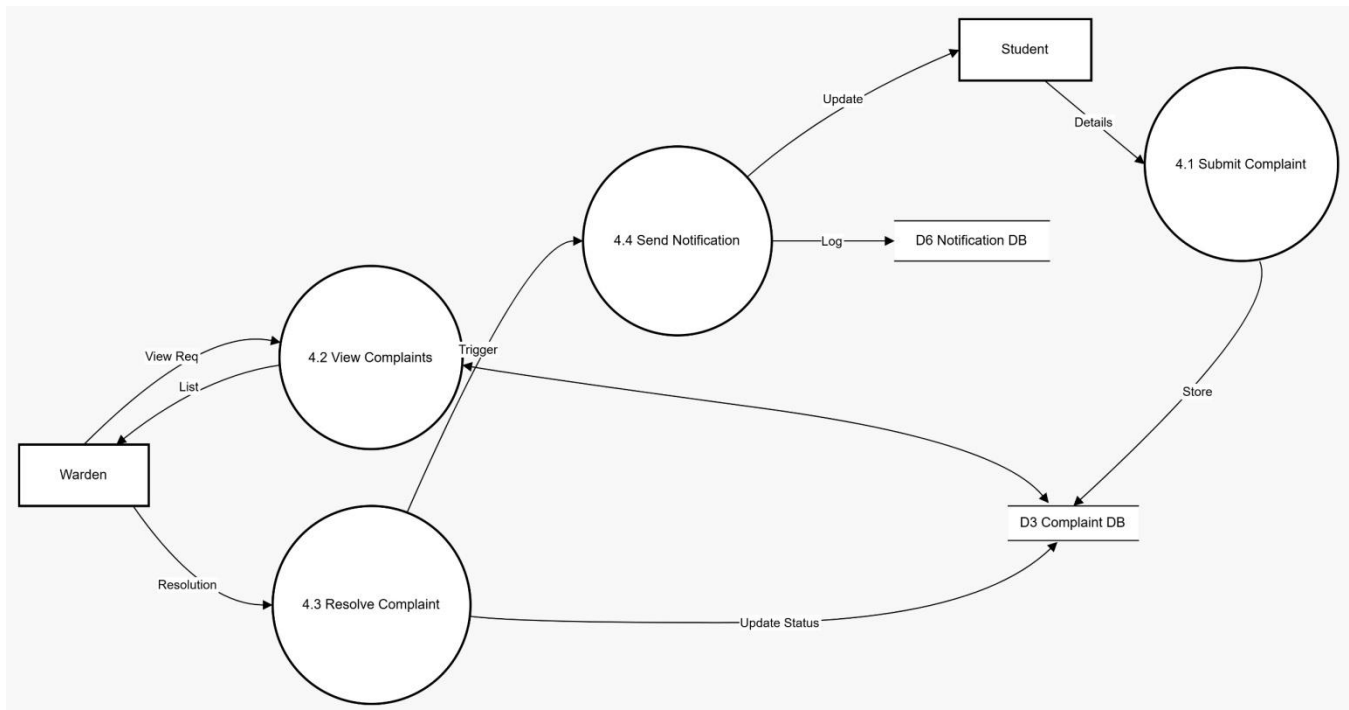
1.3.3 Process 3.0 – Hostel & Room Management



Manages the full room lifecycle from creation to vacating. Admin and Warden interact with this process; data is persisted in D1 and D2.

Sub-process	Description
3.1 Create / Update Hostel & Rooms	Admin configures new hostels and rooms, specifying capacity, rent, and amenities. Updates stored in D2.
3.2 Allocate Room	Assigns an available room to a student. Writes allocation record to D1 Student DB and updates occupancy in D2.
3.3 Track Vacancy	Continuously monitors and recalculates vacancy counts across blocks and floors, updating D2 accordingly.
3.4 Process Vacate Request	Receives student vacate request, triggers warden approval workflow, and upon approval updates room status in D2 to vacant.

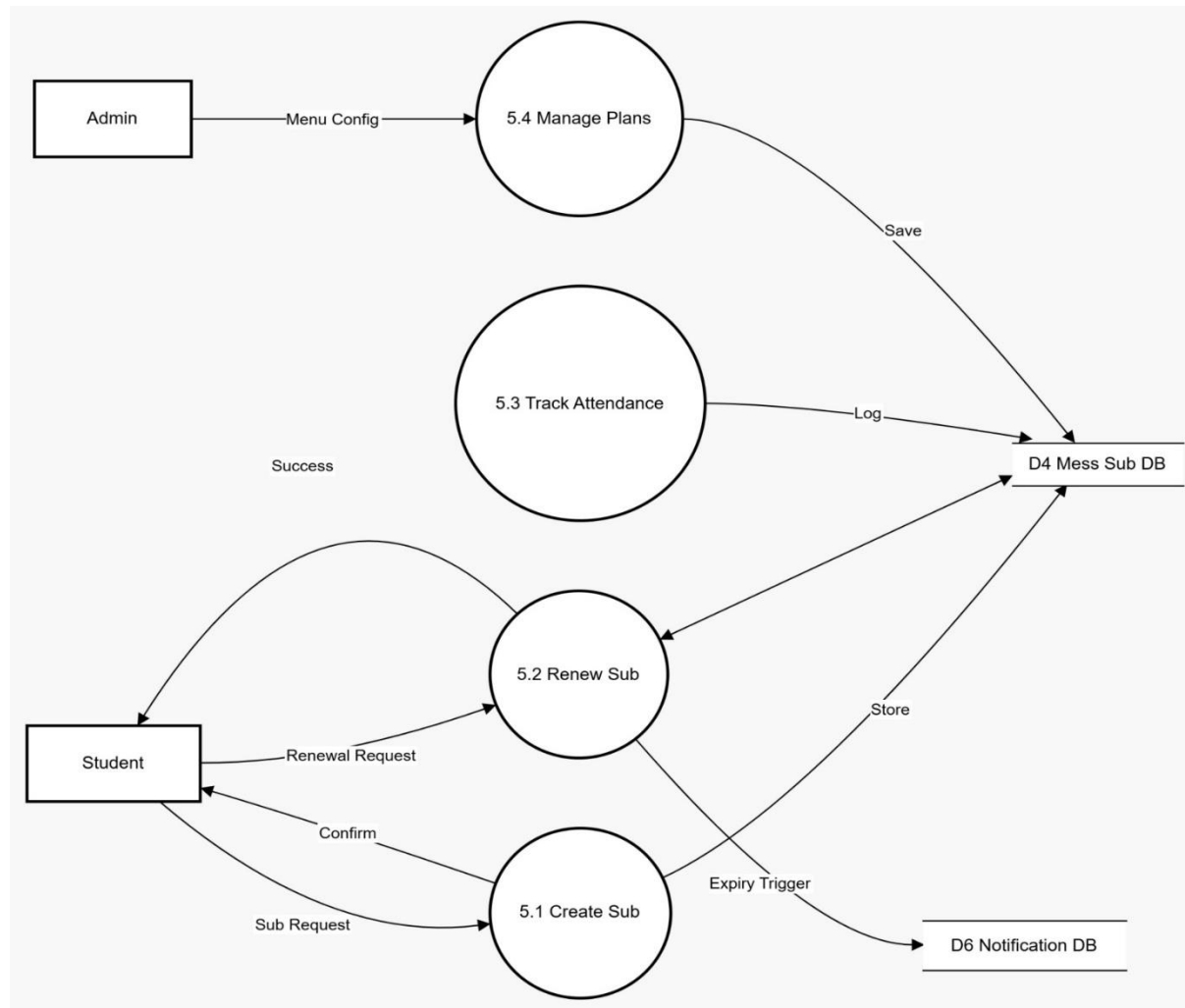
1.3.4 Process 4.0 – Complaint Management



Covers the end-to-end lifecycle of a complaint from submission to resolution. Stores data in D3 Complaint DB and sends events to D6 Notification DB.

Sub-process	Description
4.1 Submit Complaint	Student submits complaint with type, priority, and optional attachment. Record saved in D3 with status "open".
4.2 View Complaints	Warden and student can retrieve complaint lists filtered by status. Fetches records from D3.
4.3 Resolve Complaint	Warden updates complaint status to "resolved" and adds resolution notes. D3 record updated; triggers notification event.
4.4 Send Complaint Notification	On resolution, stores a notification event in D6 and dispatches status update to the affected student.

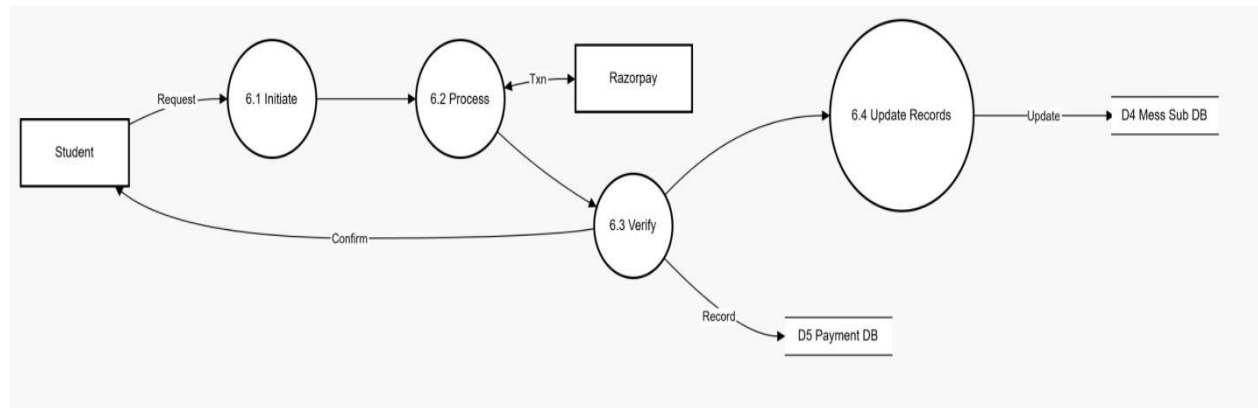
1.3.5 Process 5.0 – Mess Subscription Management



Manages subscription lifecycle, daily attendance, and menu planning. Student and Mess Admin interact with this process; data stored in D4 and D6.

Sub-process	Description
5.1 Create Subscription	Student subscribes to a mess plan (weekly/monthly). New subscription record created in D4 with expiry date.
5.2 Renew Subscription	Before or on expiry, student renews their subscription. D4 record updated with new expiry; triggers renewal notification.
5.3 Track Mess Attendance	Logs daily meal attendance per student per meal type into D4. Used for billing and analytics.
5.4 Manage Menu Plans	Mess Admin configures weekly/daily menu by meal type and food category. Menu data saved in D4.

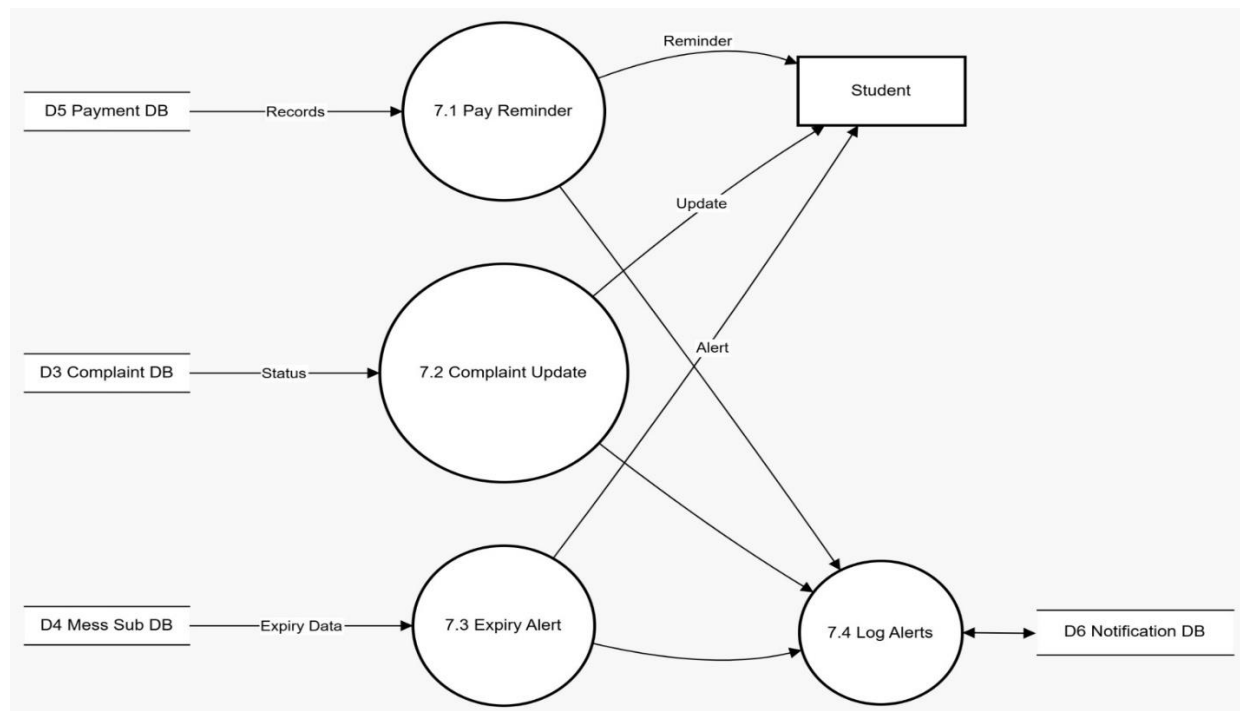
1.3.6 Process 6.0 – Payment Management



Handles the complete payment workflow including Razorpay integration, verification, and record maintenance. Interacts with D4, D5, and D6.

Sub-process	Description
6.1 Initiate Payment	Student triggers a payment request specifying type (hostel fee, mess fee, etc.) and amount. Passes details to 6.2.
6.2 Process via Razorpay	Communicates with the external Razorpay gateway, sending transaction details and receiving a payment response.
6.3 Verify Payment Status	Validates the gateway response. On success, marks transaction as "completed"; on failure, marks as "failed" and notifies student.
6.4 Update Sub / Fee Record	Persists verified transaction in D5 Payment DB. If payment was for mess subscription, updates expiry in D4. Triggers payment notification in D6.

1.3.7 Process 7.0 – Notification System



A shared utility process invoked by multiple other processes. Reads from D3, D4, D5, and writes to D6 Notification DB.

Sub-process	Description
7.1 Generate Payment Reminder	Scans D5 for upcoming due dates and generates reminder alerts for students with pending payments.
7.2 Send Complaint Update	Triggered by Process 4.3; dispatches complaint resolution status to the originating student.
7.3 Send Sub Expiry Alert	Monitors D4 for subscriptions nearing expiry and sends advance alert notifications to students.
7.4 Log & Archive Alerts	Stores all dispatched notifications in D6 for audit trail and analytics.

2. Data Dictionary

The Data Dictionary provides a precise specification of all data stores used in the Hostezy system. It defines field names, data types, constraints, keys, and relationships, serving as the authoritative reference for database schema implementation.

2.1 STUDENT

Stores academic, personal, and hostel-related information of all registered students. This is the central entity referenced by payments, complaints, attendance, and checkout records.

Business Rules:

- Email address and enrollment number must be unique across all records.
- Students with a "suspended" status cannot initiate payments or access restricted features.
- Year of study must be an integer value between 1 and 5 (inclusive).

Relationships:

- 1:N → Payment (a student may have many payment records)
- 1:N → Complaint (a student may raise many complaints)
- 1:N → Mess_Attendance (daily meal attendance per student)
- 1:1 → Checkout (one checkout event per student)
- N:1 → Room (many students may share a room)

Index Recommendations:

- Index on college_email – supports fast login lookup.
- Index on enrollment_number – used in admin search.
- Index on status – filters active/suspended students efficiently.

Field Name	Data Type	Size	PK	FK	Nullable	Default	Constraints
student_id	INT	—	☒	☐	☐	AUTO_INCREMENT	Primary Key
first_name	VARCHAR	50	☐	☐	☐	—	—
last_name	VARCHAR	50	☐	☐	☐	—	—
college_email	VARCHAR	100	☐	☐	☐	—	UNIQUE
phone_number	VARCHAR	15	☐	☐	☐	—	—
parent_guardian_name	VARCHAR	100	☐	☐	☐	—	—
parent_guardian_contact	VARCHAR	15	☐	☐	☐	—	—
enrollment_number	VARCHAR	20	☐	☐	☐	—	UNIQUE
course	VARCHAR	50	☐	☐	☐	—	—
year_of_study	TINYINT	—	☐	☐	☐	—	CHECK (1–5)
department	VARCHAR	50	☐	☐	☐	—	—
admission_date	DATE	—	☐	☐	☐	CURRENT_DATE	—
status	ENUM	—	☐	☐	☐	'active'	(active, inactive, suspended)

2.2 WARDEN

Stores details of hostel wardens responsible for managing specific hostel blocks. Wardens can view complaints, approve vacate requests, and manage room allocations.

Business Rules:

- Each warden must have a unique employee ID.
- A warden is assigned to exactly one hostel block at a time.

Relationships:

- 1:N → Room (a warden oversees multiple rooms)
- 1:N → Complaint (complaints are assigned to a warden for resolution)

Index Recommendations:

- Index on employee_id – used in authentication and admin lookup.

Field	Type	Size	PK	FK	Nullable	Constraints
warden_id	INT	—	☐	☐	☐	AUTO_INCREMENT
first_name	VARCHAR	50	☐	☐	☐	—
last_name	VARCHAR	50	☐	☐	☐	—
phone_number	VARCHAR	15	☐	☐	☐	—
date_of_joining	DATE	—	☐	☐	☐	—
hostel_block_assigned	VARCHAR	20	☐	☐	☐	—
employee_id	VARCHAR	20	☐	☐	☐	UNIQUE

2.3 HOSTEL_ADMIN

Stores administrative authority details for hostel system administrators. Admins have the highest access level and can manage wardens, rooms, students, and generate reports.

Business Rules:

- Access level determines system privileges: full, limited, or read_only.
- College email must be unique — used as the login identifier.

Relationships:

- 1:N → Room (admin configures rooms)
- 1:N → Warden (admin manages warden assignments)

Field	Type	PK	Nullable	Constraints
admin_id	INT	☐	☐	AUTO_INCREMENT
first_name	VARCHAR(50)	☐	☐	—
last_name	VARCHAR(50)	☐	☐	—
college_email	VARCHAR(100)	☐	☐	UNIQUE
phone_number	VARCHAR(15)	☐	☐	—
designation	VARCHAR(50)	☐	☐	—
department	VARCHAR(50)	☐	☐	—

Field	Type	PK	Nullable	Constraints
access_level	ENUM(full, limited, read_only)	☐	☐	—

2.4 MESS_ADMIN

Stores details of mess administrators responsible for managing menu plans, food type configuration, and mess operational decisions.

Business Rules:

- Employee ID and email must be unique.
- mess_type_managed defines whether the admin oversees veg, non-veg, or both mess operations.

Relationships:

- 1:N → Mess_Menu (mess admin creates and manages menu entries)

Field	Type	PK	Constraints
mess_admin_id	INT	☐	AUTO_INCREMENT
first_name	VARCHAR(50)	☐	—
last_name	VARCHAR(50)	☐	—
email	VARCHAR(100)	☐	UNIQUE
phone_number	VARCHAR(15)	☐	—
employee_id	VARCHAR(20)	☐	UNIQUE
mess_type_managed	ENUM(veg, non-veg, both)	☐	—

2.5 PAYMENT

Records all financial transactions initiated by students including hostel fees, mess fees, security deposits, and fines. Integrates with Razorpay for online transactions.

Business Rules:

- Amount must be greater than zero.
- Transaction ID must be unique to prevent duplicate payment records.
- Payment for month must be a valid month number between 1 and 12.

Relationships:

- N:1 → Student (many payments belong to one student)

Index Recommendations:

- Index on student_id – supports fast retrieval of student payment history.
- Composite index on (student_id, payment_for_month, payment_for_year) – used for monthly due checks.

Field	Type	PK	FK	Constraints
payment_id	INT	☐	☐	AUTO_INCREMENT
student_id	INT	☐	☐	FK → Student

Field	Type	PK	FK	Constraints
payment_type	ENUM(hostel_fee, mess_fee, security_deposit, fine)	☐	☐	—
amount	DECIMAL(10,2)	☐	☐	> 0
payment_date	DATE	☐	☐	—
payment_method	ENUM(cash, card, upi, net_banking)	☐	☐	—
transaction_id	VARCHAR(100)	☐	☐	UNIQUE
payment_for_month	TINYINT	☐	☐	CHECK(1–12)
payment_for_year	SMALLINT	☐	☐	—
status	ENUM(pending, completed, failed, refunded)	☐	☐	—

2.6 MESS_MENU

Stores daily and weekly menu information for each meal type. Allows mess admin to configure dishes, calorie information, allergen details, and availability.

Business Rules:

- Each menu entry is uniquely identified by date, meal type, and food type combination.
- Availability status controls whether the dish appears on the student-facing menu.

Relationships:

- N:1 → Mess_Admin (menu entries are created by a mess admin)

Field	Type	PK	FK
menu_id	INT	☐	☐
date	DATE	☐	☐
meal_type	ENUM(breakfast, lunch, snacks, dinner)	☐	☐
food_type	ENUM(veg, non-veg, jain)	☐	☐
dish_name	VARCHAR(100)	☐	☐
description	TEXT	☐	☐
items_included	TEXT	☐	☐
calories	INT	☐	☐
allergen_info	TEXT	☐	☐
availability_status	ENUM(available, not_available)	☐	☐
created_by	INT	☐	FK → Mess_Admin
created_date	TIMESTAMP	☐	DEFAULT CURRENT_TIMESTAMP
updated_date	TIMESTAMP	☐	ON UPDATE CURRENT_TIMESTAMP

2.7 COMPLAINT

Tracks all complaints raised by students, including type, priority, assignment, and resolution details. Supports the full complaint lifecycle from submission to closure.

Business Rules:

- Feedback rating must be between 1 and 5 if provided.
- resolved_date must be set when status transitions to "resolved" or "closed".

Relationships:

- N:1 → Student (complaint belongs to a student)
- N:1 → Staff/Warden/Mess_Admin (complaint assigned to a resolver)

Field	Type	PK	FK
complaint_id	INT	☐	☐
student_id	INT	☐	FK → Student
complaint_type	ENUM(room, mess, maintenance, cleanliness, staff, other)	☐	☐
category	ENUM(hostel, mess)	☐	☐
subject	VARCHAR(200)	☐	☐
priority	ENUM(low, medium, high, urgent)	☐	☐
status	ENUM(open, in_progress, resolved, closed)	☐	☐
filed_date	DATE	☐	☐
assigned_to	INT	☐	FK → Staff/Warden/Mess_Admin
resolved_date	DATE	☐	Nullable
attachments	TEXT	☐	Nullable
feedback_rating	TINYINT	☐	CHECK(1–5)

2.8 CHECKOUT

Records student checkout events including reason, room condition assessment, damage charges, pending dues, and security deposit refund calculation.

Business Rules:

- A student can only have one checkout record (1:1 relationship with Student).
- security_deposit_refund = deposit paid – damage_charges – pending_dues.
- damage_charges and pending_dues default to zero if no issues are found.

Relationships:

- 1:1 → Student
- N:1 → Room (checkout releases the room back to vacancy)

Field	Type	PK	FK
checkout_id	INT	☐	☐
student_id	INT	☐	FK → Student
room_id	INT	☐	FK → Room
checkout_date	DATE	☐	☐
checkout_reason	ENUM(course_completion, transfer, personal, expelled)	☐	☐
room_condition	TEXT	☐	☐
damage_charges	DECIMAL(10,2)	☐	DEFAULT 0
pending_dues	DECIMAL(10,2)	☐	DEFAULT 0
security_deposit_refund	DECIMAL(10,2)	☐	—

2.9 STAFF

Stores operational staff details including cleaners, cooks, security personnel, and maintenance workers. Staff members can be assigned to complaints and are managed by admins.

Business Rules:

- Each staff member must have a unique employee ID (derived from staff_id and designation).
- Salary and id_proof_type are mandatory for compliance and HR records.

Relationships:

- N:1 → Department (hostel, mess, or maintenance)
- Can be assigned to Complaint records as resolvers

Field	Type	PK
staff_id	INT	☐
first_name	VARCHAR(50)	☐
last_name	VARCHAR(50)	☐
email	VARCHAR(100)	UNIQUE
phone_number	VARCHAR(15)	☐
designation	ENUM(cleaner, cook, security, maintenance)	☐
department	ENUM(hostel, mess, maintenance)	☐
date_of_joining	DATE	☐
shift_timing	VARCHAR(50)	☐
salary	DECIMAL(10,2)	☐
assigned_block_floor	VARCHAR(50)	☐
id_proof_type	VARCHAR(50)	☐

2.10 ROOM

Defines the physical structure of hostel rooms, including type, capacity, occupancy status, rent, and amenities. Central to allocation and vacancy tracking.

Business Rules:

- current_occupancy must never exceed capacity.
- room_status transitions: vacant → occupied (on allocation), occupied → vacant (on checkout), any → under_maintenance (admin action).

Relationships:

- 1:N → Student (room accommodates multiple students up to capacity)
- N:1 → Warden (each room is overseen by one warden)

Field	Type	PK	FK
room_id	INT	☐	☐
room_number	VARCHAR(10)	☐	☐
block_name	VARCHAR(20)	☐	☐
floor_number	INT	☐	☐
room_type	ENUM(single, double, triple, quad)	☐	☐
capacity	INT	☐	☐
current_occupancy	INT	☐	DEFAULT 0
room_status	ENUM(vacant, occupied, under_maintenance)	☐	☐
rent_per_bed	DECIMAL(10,2)	☐	☐
amenities	TEXT	☐	☐
assigned_warden_id	INT	☐	FK → Warden
remarks	TEXT	☐	Nullable

2.11 MESS_ATTENDANCE

Logs daily meal attendance for each student per meal type. Used for billing calculations, subscription utilisation analysis, and mess demand forecasting.

Business Rules:

- Composite unique constraint on (student_id, date, meal_type) prevents duplicate attendance entries.
- Attendance can only be logged for students with an active mess subscription.

Relationships:

- N:1 → Student (many attendance records per student)

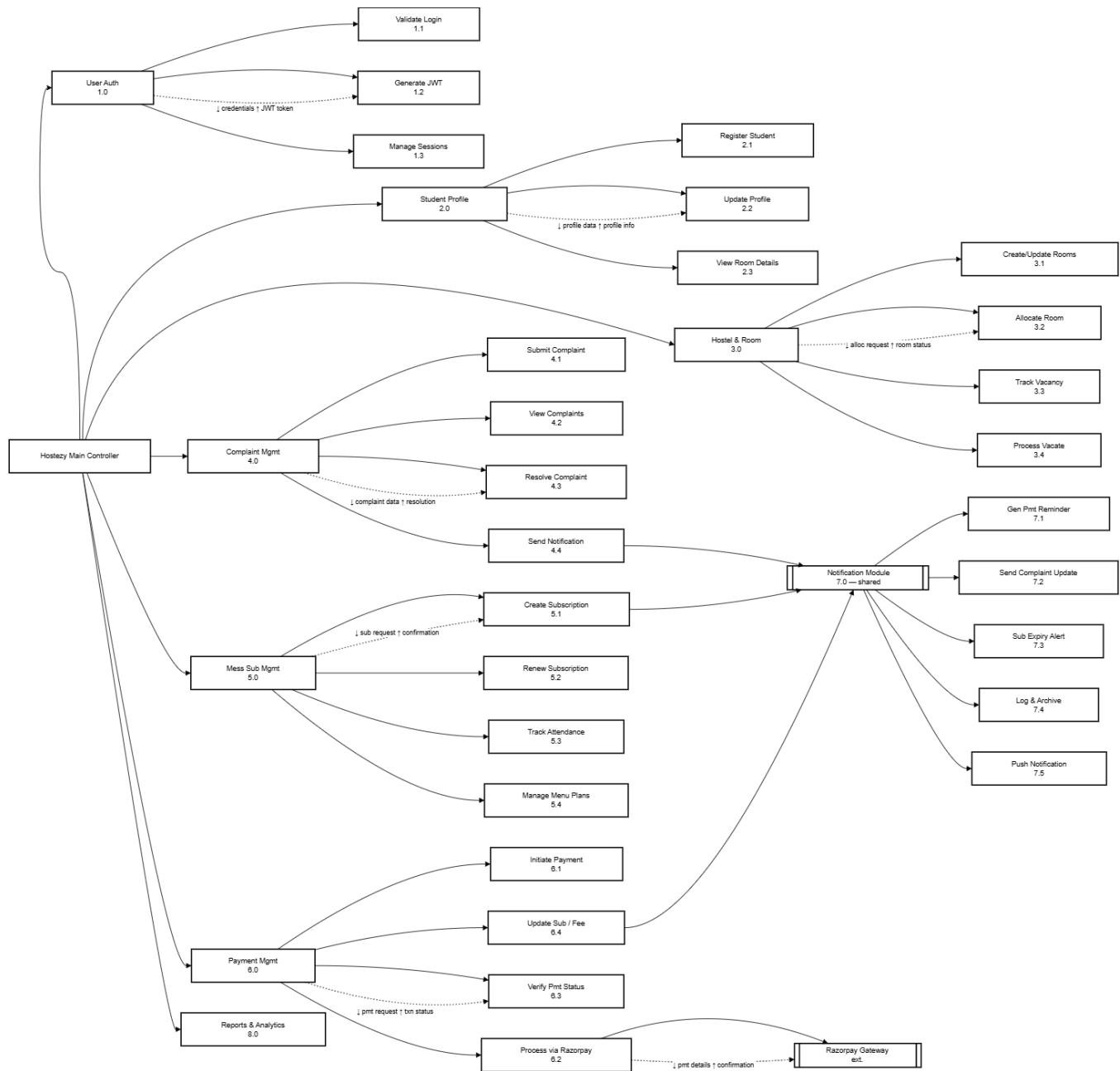
Index Recommendations:

- Composite index on (student_id, date) – supports fast retrieval of a student's attendance history for a date range.

Field	Type	PK	FK
attendance_id	INT	☐	☐
student_id	INT	☐	FK → Student
date	DATE	☐	☐
meal_type	ENUM(breakfast, lunch, dinner)	☐	☐
attendance_status	ENUM(present, absent)	☐	☐

3. Module Structure Chart (MSC)

The Module Structure Chart represents the hierarchical decomposition of the Hostezy system following the principles of structured design. It shows the calling hierarchy between modules, the data couples (parameters exchanged), and the control flow from the Main Controller down to leaf-level sub-processes. The notation follows Yourdon-Constantine structured design conventions.



3.1 Notation Guide

Symbol	Meaning
Solid rectangle	Internal module – a functional unit implemented within the system.
Dashed rectangle	Shared / utility module – called by multiple parent modules (e.g. Notification Module 7.0).
Double-bordered rectangle	External module – a component outside the system boundary (e.g. Razorpay Gateway).
Solid line (parent → child)	Module call – the parent invokes the child module.
Dashed line with filled circle	Data couple – parameter passed between caller and callee; circle at the source end, arrowhead shows direction.

3.2 Module Descriptions

MC Hostezy Main Controller

The root module. Orchestrates the entire application by delegating control to each of the seven functional subsystems. Receives the authenticated user context and routes requests accordingly. Does not implement business logic directly — it acts purely as a dispatcher.

1.0 User Auth

Validates user credentials against D1 Student DB, generates a signed JWT token encoding user role and expiry, and maintains session state. Called on every login attempt. Returns a JWT token to the calling portal on success, or an error flag on failure.

Data Couples: *Input: username, password. Output: JWT token, session ID.*

2.0 Student Profile

Manages student profile data — registration, profile updates, and room detail retrieval. Reads from D1 Student DB and D2 Hostel DB. Invoked from both the Student Portal and the Admin Dashboard.

Data Couples: *Input: profile data, room ID. Output: profile info, room details.*

3.0 Hostel & Room Mgmt

Controls the full room lifecycle. Admin creates and configures rooms (3.1), the system allocates rooms to students (3.2), tracks real-time vacancy (3.3), and processes vacate requests through an approval workflow (3.4).

Data Couples: *Input: allocation request, vacate request. Output: room status, confirmation.*

4.0 Complaint Mgmt

Handles complaint submission (4.1), display (4.2), resolution (4.3), and outbound notification triggers (4.4). Calls the shared Notification Module 7.0 on status changes.

Data Couples: *Input: complaint data, resolution details. Output: complaint list, resolution status.*

5.0 Mess Sub Mgmt

Manages mess subscription creation and renewal (5.1/5.2), daily attendance tracking (5.3), and menu plan configuration by Mess Admin (5.4). Calls Notification Module 7.0 on subscription expiry approach.

Data Couples: *Input: sub request, menu config. Output: confirmation, attendance record.*

6.0 Payment Mgmt

Orchestrates the payment pipeline: student initiates (6.1), Razorpay processes (6.2), system verifies (6.3), and subscription/fee records are updated (6.4). Calls external Razorpay Gateway module and shared Notification Module 7.0.

Data Couples: *Input: payment request, amount. Output: txn status, confirmation.*

7.0 Notification Module (shared)

A cross-cutting utility module invoked by Complaint Mgmt (4.4), Mess Sub Mgmt (5.1), and Payment Mgmt (6.4). Generates payment reminders (7.1), complaint updates (7.2), sub expiry alerts (7.3), and archives all dispatched alerts (7.4).

Data Couples: *Input: event trigger. Output: notification record, alert log.*

8.0 Reports & Analytics

Called directly by the Main Controller on admin/warden dashboard load. Aggregates data across D1–D5 to produce occupancy reports, payment summaries, complaint analytics, and mess utilisation statistics.

Data Couples: *Input: query parameters (date range, filter). Output: aggregated report data.*

6.E Razorpay Gateway (External)

An external module representing the Razorpay payment gateway. Called exclusively by Process 6.2 (Process via Razorpay). Receives payment details and returns a transaction response. The double-border notation denotes its external nature — it is outside the system's direct control.

Data Couples: *Input: payment details. Output: transaction confirmation / failure code.*

4. Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) maps each functional and non-functional requirement from the Software Requirements Specification (SRS) to the corresponding design artefacts in this document. A ✓ indicates that the requirement is addressed by that artefact. The matrix ensures complete design coverage and supports verification and validation activities.

Artefact columns are grouped into three categories: Data Flow Diagrams (DFDs 1.0–7.0), Data Dictionary tables (DD), and Module Structure Chart nodes (MSC). A requirement must be traceable to at least one artefact in each category to be considered fully covered.

Req. ID	Requirement Description	Data Flow Diagrams							Data Dictionary					Module Structure Chart				
Req. ID	Description	DFD 1.0 User Auth	DFD 2.0 Student Profile	DFD 3.0 Hostel & Room	DFD 4.0 Complaint	DFD 5.0 Mess Sub	DFD 6.0 Payment	DFD 7.0 Notification	DD Student	DD Room	DD Payment	DD Complaint	DD Mess Menu	MSC 1.0 Auth	MSC 3.0 Hostel	MSC 5.0 Mess	MSC 6.0 Pmt	MSC 7.0 Notif
FR-01	User registration & authentication	✓							✓					✓				
FR-02	JWT-based session management	✓												✓				
FR-03	Student profile management		✓						✓									
FR-04	Room & hostel allocation			✓						✓					✓			
FR-05	Vacancy tracking & vacate request			✓						✓					✓			
FR-06	Complaint registration				✓							✓						
FR-07	Complaint resolution & status update				✓							✓						
FR-08	Mess subscription (create / renew)					✓							✓			✓		
FR-09	Mess attendance tracking					✓							✓			✓		
FR-10	Mess menu management					✓							✓			✓		
FR-11	Payment initiation via Razorpay						✓				✓						✓	
FR-12	Payment verification & record						✓				✓						✓	
FR-13	Payment reminder notifications							✓										✓
FR-	Subscription							✓										✓

Req. ID	Requirement Description	Data Flow Diagrams							Data Dictionary					Module Structure Chart				
Req. ID	Description	DFD 1.0 User Auth	DFD 2.0 Student Profile	DFD 3.0 Hostel & Room	DFD 4.0 Complaint	DFD 5.0 Mess Sub	DFD 6.0 Payment	DFD 7.0 Notification	DD Student	DD Room	DD Payment	DD Complaint	DD Mess Menu	MSC 1.0 Auth	MSC 3.0 Hostel	MSC 5.0 Mess	MSC 6.0 Pmt	MSC 7.0 Notif
14	expiry alerts																	
FR-15	Admin & warden dashboards		✓	✓	✓	✓	✓											
FR-16	Reports & analytics generation			✓	✓	✓	✓											
FR-17	Student checkout processing			✓						✓					✓			
NFR-01	System response < 2 seconds	✓					✓	✓						✓			✓	✓
NFR-02	JWT + HTTPS security	✓					✓				✓			✓			✓	✓
NFR-03	99.9% system availability	✓	✓	✓	✓	✓	✓	✓						✓	✓	✓	✓	✓

Symbol	Meaning
✓	Requirement is covered by this design artefact.
	No direct traceability to this artefact.
NFR	Non-Functional Requirement – traced to architectural/cross-cutting design decisions.

Software Requirements Specification (SRS)

Hostezy: Smart Hostel & Mess Management System

Date: 15th February 2026

1. Introduction

1.1. Purpose

This document describes the **software requirements** for the **Hostezy: Smart Hostel & Mess Management System**.

The purpose of this SRS is to provide a **clear and complete description** of the system's functional and non-functional requirements for developers, testers, faculty evaluators, and stakeholders.

1.2. Scope

The Smart Hostel & Mess Management System is a **web-based platform** designed to automate and manage hostel and mess operations efficiently.

- Student registration and authentication
- Hostel, room, and occupancy management
- Complaint and issue tracking
- Mess management with subscription-based access (weekly/monthly)
- Secure online payments
- Automated payment reminder before subscription expiry
- Admin and warden dashboards
- Notifications, reports, and analytics

1.3. Intended Audience

Stakeholders, developers, testers, project guide and evaluators, system administrators, and hostel operations personnel.

1.4. Product Features

- User authentication (Student, Admin, Warden)
- Student profile management
- Hostel, room, and allocation management
- Complaint registration and resolution
- Mess attendance and menu management
- Subscription-based mess access (weekly / monthly)
- Payment and fee management using Razorpay
- Automated payment reminders before subscription expiry
- Admin and warden dashboards
- Notification and communication system
- Reports and analytics

1.5. Non-Functional Requirements

- Performance: System responds within 2 seconds for standard operations
- Security & Privacy: JWT authentication, role-based access, HTTPS, secure payment gateway
- Maintainability: Modular and well-documented codebase
- Usability: Responsive UI for desktop and mobile
- Reliability: 99.9% system availability
- Scalability: Supports increasing number of users and hostels

1.6. Architecture Overview

- Frontend: React
- Backend: Node.js, Express.js
- Database: MongoDB
- Authentication: JWT
- Payment Gateway: Razorpay
- Hosting: Vercel

1.7. Future Enhancements

- Mobile application support
- AI-based mess demand prediction
- Biometric-based attendance
- Smart IoT-based room monitoring
- Integration with institute ERP

1.8. User Stories for Hostezy: Smart Hostel & Mess Management System

ID	User Role	Wants To	So That
US001	Student	register and login	access hostel services securely
US002	Student	view my room and hostel details	know my accommodation status
US003	Student	raise a complaint	report hostel-related issues
US004	Warden	view and resolve complaints	maintain hostel discipline
US005	Admin	manage hostels and rooms	control infrastructure
US006	Student	subscribe to mess weekly/monthly	access mess services
US007	Student	renew mess subscription online	continue mess access
US008	System	send payment reminder before expiry	avoid service interruption
US009	Student	pay fees online	complete transactions easily
US010	Student	view payment history	track my expenses
US011	Admin	manage mess plans	control pricing and duration
US012	Admin	generate reports	analyze hostel usage
US013	Warden	approve vacate requests	manage room availability
US014	Student	receive notifications	stay informed
US015	Admin	manage users and roles	maintain system control

1.9. Hostezy: Smart Hostel & Mess Management System - Project Plan

Overview

- Project Duration: Approximately 8–10 weeks
- Development Methodology: Agile (Scrum with weekly sprints)

Phase 1: Requirement Analysis & Planning (Week 1)

Deliverables:

- Finalized SRS
- Feature prioritization
- Technology stack decision
- Project roadmap

Tasks:

- Stakeholder discussion
- Finalization of user stories
- Project planning

Phase 2: Design (Week 2)

Deliverables:

- UI wireframes
- Database schema
- System architecture

Tasks:

- Dashboard design
- User flow design

Phase 3: Backend Development (Week 3–4)

Deliverables:

- REST APIs
- JWT authentication
- Database integration

Tasks:

- User, hostel, mess modules
- Payment integration
- Notification service

Phase 4: Frontend Development (Week 5–6)

Deliverables:

- Student portal
- Admin dashboard
- Warden dashboard

Tasks:

- API integration
- Responsive UI

Phase 5: Testing (Week 7)

Deliverables:

- Test cases
- Bug fixes

Tasks:

- Functional testing
- Security testing

Phase 6: Deployment (Week 8)

Deliverables:

- Cloud deployment
- Production configuration

Phase 7: Final Submission (Week 9)

Deliverables:

- Final project demo
- Documentation

Tools & Technologies

Area	Tools / Stack
Frontend	React
Backend	Node.js, Express
Database	MongoDB
Payments	Razorpay
Auth	JWT
DevOps	GitHub
Design	Figma
Testing	Postman

Timeline Overview

Phase	Duration (Weeks)	Sprint(s)
Req. Analysis & Planning	1	Sprint 1
Design	1	Sprint 2
Backend Development	2	Sprint 3–4
Frontend Development	2	Sprint 5–6
Testing	1	Sprint 7
Deployment	1	Sprint 8
Final Submission	1	Sprint 9